



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Tired of compatibility headache... let's try containers.

Date: 04 June 2026

Jijo Derick Abraham

(Postdoctoral research fellow)

Computational Multiphysics Laboratory (CML),

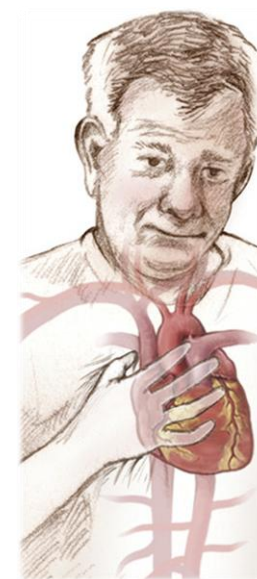
School of Mechanical and Mining Engineering, The University of Queensland.

About me and my research

Jijo Derick Abraham

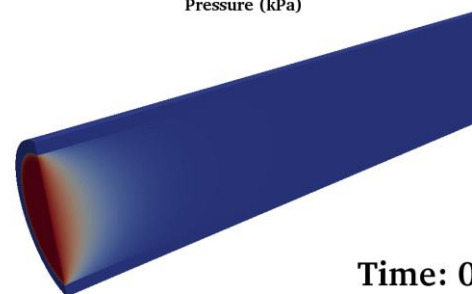
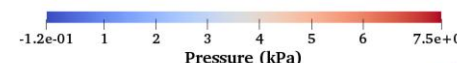
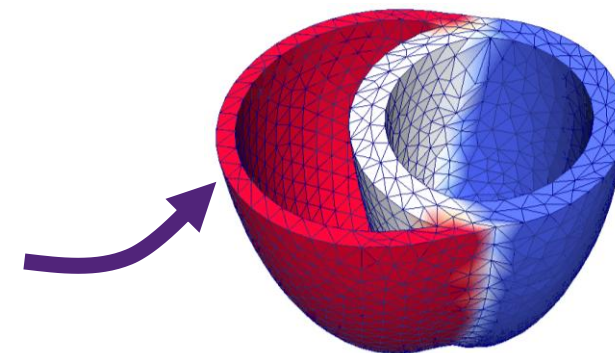
- 2026-present : **Postdoctoral research fellow**
University of Queensland, Australia,
- 2020-2026 : **Joint PhD, Computational cardiac mechanics**
University of Queensland, Australia, &
Indian Institute of Technology Delhi.
- 2019-2020 : **Junior Reserch Fellow**
(FSI solvers using OpenFOAM)
Visvesvaraya National Institute of Technology, Nagpur India
- 2017-2019 : **M.Tech, Heat Power Engineering**
Visvesvaraya National Institute of Technology, Nagpur India
- 2012-2016 : **B.Tech, Mechanical Engineering**
Cochin University of Science and Technology, India

*Happy to connect via
Linkedin, Github*

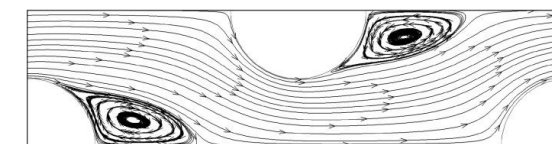


*Rosalia et al. 2021,
JACC Basic Transl Sci*

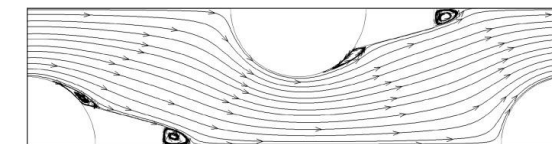
Heart modelling



Time: 0.1 ms



(a) Bare tubes



(b) L/D=0.50



CRICOS code 00025B

- Why we need container?
- What is a container?
- How does the container work?
- Types of containers
- Working with docker
- Working with singularity
- Drawbacks of containers

Why we (CFD researchers) need a containers?

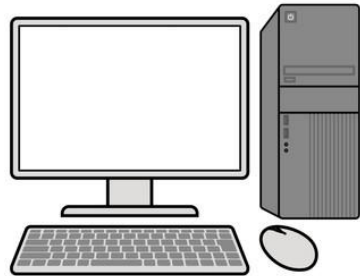
Situations in which we face compatibly headache

- Trying a new/old code in your field
- Transferring your code to a new member with a different machine
- Want to run your code in a new machine or cluster

Works on one machine, but not on others



Laptop
(Different OS)



Office PC
(Different Setup)



Cluster
HPC/Cloud
(Different Env.)

Dependency conflicts
Inconsistent environments
Hard to reproduce results
Time wasted on setup & debugging

Low Productivity, Not reproducible, Hard to scale

How container solves the compatibility headache?

Your CFD application/solver code
System libraries
Dependencies
System tools
Runtime
Others



Contain package your CFD code with
everything it need to run consistently,
anywhere

With containers: build once, run anywhere

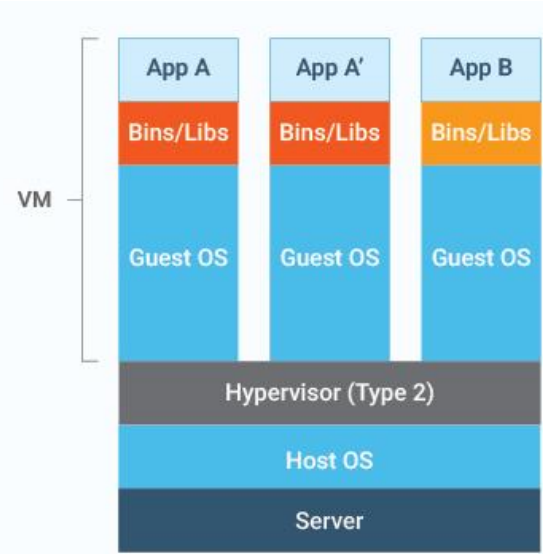


Cluster

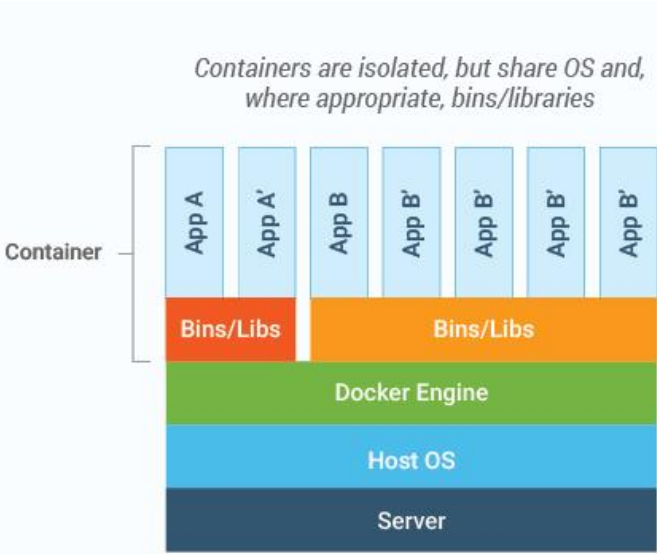
- Consistent environment
- No dependency conflicts
- Portable
- Reproducible results
- Save time and boost productivity

How does containers work?

Virtual machines



Containers



Virtual machine

Containers

Runs on its own OS

Share host OS

Slow boot (minutes)

Fast boot (milliseconds)

Limited performance

Native performance

Large image size

Small image size

Require more memory

Require less memory

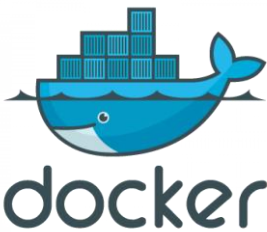
Hard to use more VMs in one machine

Easy to use many containers in one machine

More secure. Fully isolated

Less secure. Process level security

<https://www.eginnovations.com/blog/containers-vs-vms/>



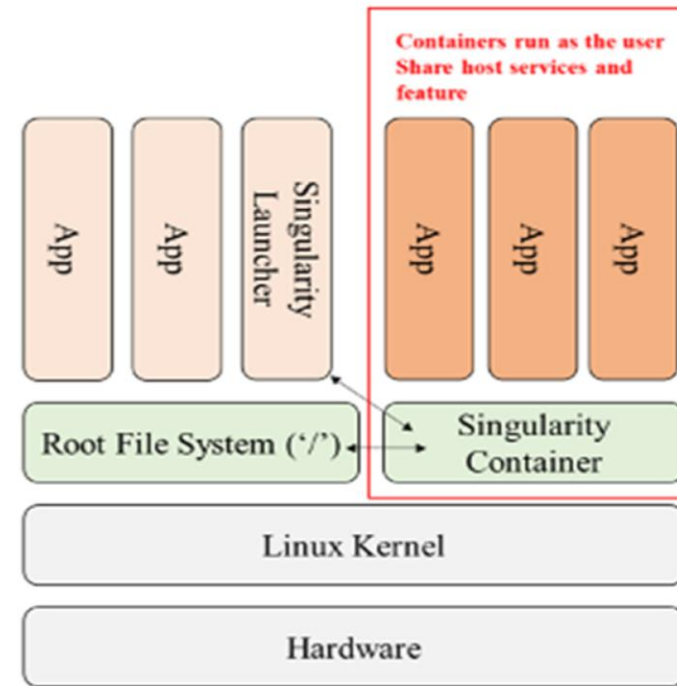
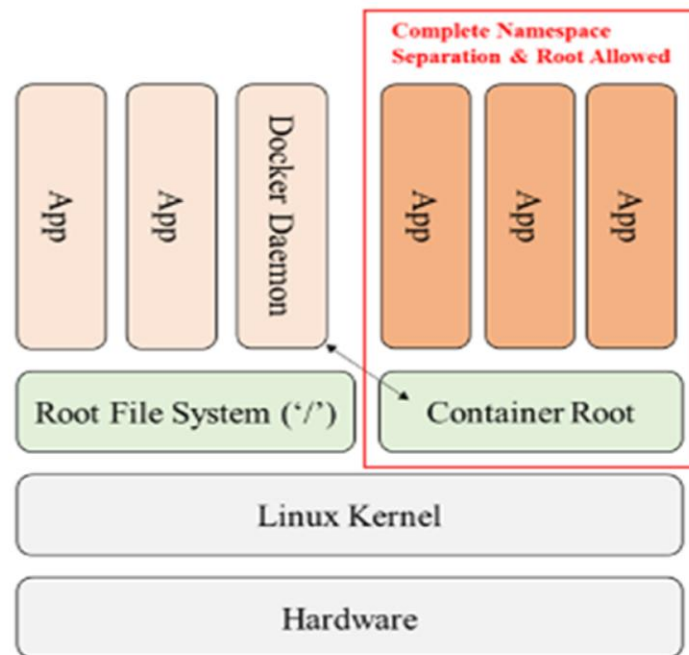
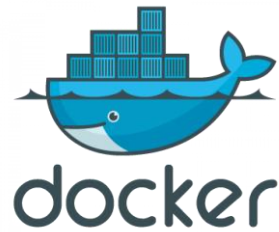
podman



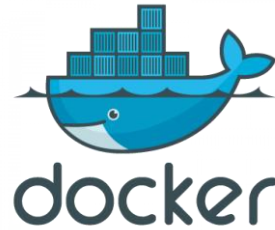
APPTAINER

<https://ipccisco.com/lesson/container-vs-virtual-machine/>

Types of containers



Comparison of containers

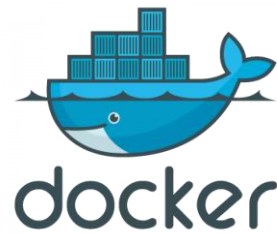


Feature	Docker	Singularity (APPTAINER)
Security	Less secure as runs at root in default	More secure
User access	Requires root privileges	Users can directly. No root privilege required.
Best for	Development, testing, local environments	HPC, research computing, shared clusters
Performance	Excellent	Better on HPC
Image format	Layered images	Single file images

Take away: Use docker for local development, and use singularity for secure, reproducible and HPC environments

Part -A

Docker workflow



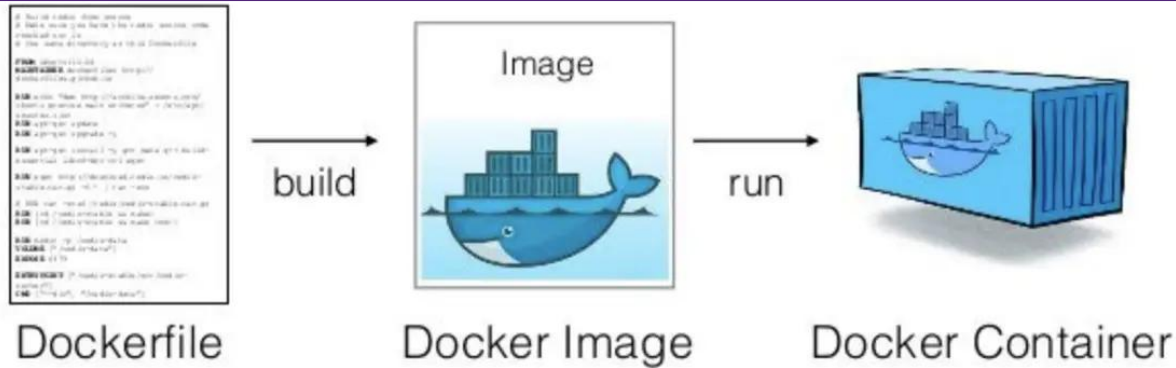
Steps of making a container

To install docker engine (<https://docs.docker.com/engine/install/>)

```
sudo apt install docker.io
```

Check the status

```
sudo systemctl status docker
```



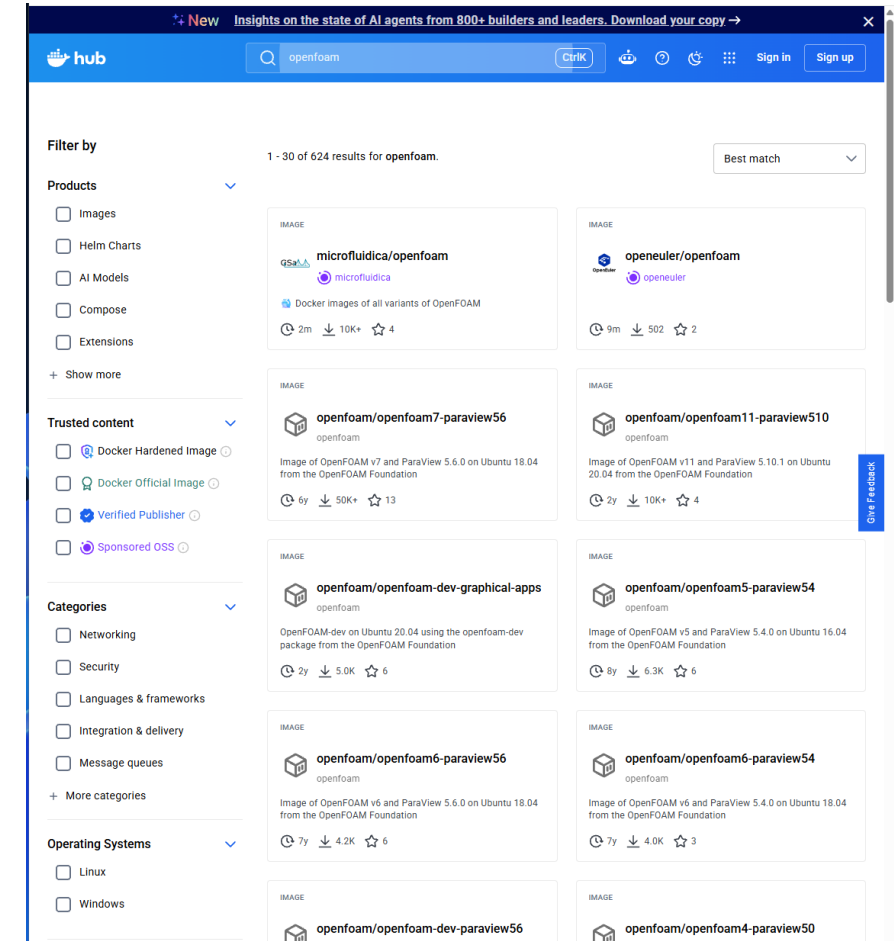
<https://www.mlwithramin.com/blog/docker>

To download the image from docker image repositories such as,
Docker Hub, Quay, Biocontainers, Rocker, Jupyter containers,
GitHub Container Registry, and Gitlab Container Registry

To pull an image

```
sudo docker pull image repository
```

Eg: `sudo docker pull fenics/stable:py3`



<https://hub.docker.com/>

Building a docker image from Dockerfile

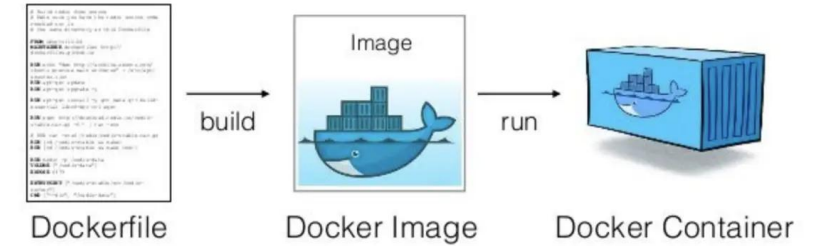
Prepare a docker file in the working directory. Save it as “Dockerfile”

Base image.
Other wise use
scratch

RUN =>
Commands to
executed while
building the
image

CMD => Commands to
executed after making the
container out of the image

```
E: > CFDTEA_presentation > Demonstration_files > Demonstration_files >  
1 FROM dolfinx/dolfinx:stable  
2  
3 RUN apt-get update && \  
4 apt-get install -y nano && \  
5 rm -rf /var/lib/apt/lists/*  
6  
7 RUN pip install --no-cache-dir matplotlib  
8  
9 CMD ["/bin/bash"]  
10  
11
```



<https://www.mlwithramin.com/blog/docker>

To build an image from the docker file

```
Sudo docker build -t <imagename> .
```

List local images

- `sudo docker images`

Docker file of openFOAM

Base image
Other wise use
scratch

RUN=Comma
nds to
executed while
building the
image

```
simple-openfoam-dockerfile / openfoam.org-release / Dockerfile
robinknowles Updated to accompany OnCFD Issue 009 - re-organised into dirs & updat...
Code Blame 25 lines (20 loc) · 694 Bytes
1 FROM ubuntu:focal
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update \
6     && apt-get install -y \
7         vim \
8         ssh \
9         sudo \
10        wget \
11        software-properties-common ;\
12        rm -rf /var/lib/apt/lists/*
13
14 RUN useradd --user-group --create-home --shell /bin/bash foam ;\
15     echo "foam ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
16
17 RUN sh -c "wget -O - http://dl.openfoam.org/gpg.key | apt-key add -" ;\
18     add-apt-repository http://dl.openfoam.org/ubuntu ;\
19     apt-get update ;\
20     apt-get install -y openfoam8 paraviewopenfoam56- ;\
21     rm -rf /var/lib/apt/lists/* ;\
22     echo "source /opt/openfoam8/etc/bashrc" >> ~foam/.bashrc ;\
23     echo "export OMPI_MCA_btl_vader_single_copy_mechanism=none" >> ~foam/.bashrc
24
25 USER foam
```

To build an image from the docker file

`Sudo docker build -t imagefile`

List local images

- `sudo docker images`

More options in the Dockerfile

Instruction	Description
<u>ADD</u>	Add local or remote files and directories.
<u>ARG</u>	Use build-time variables.
<u>CMD</u>	Specify default commands.
<u>COPY</u>	Copy files and directories.
<u>ENTRYPOINT</u>	Specify default executable.
<u>ENV</u>	Set environment variables.
<u>EXPOSE</u>	Describe which ports your application is listening on.
<u>FROM</u>	Create a new build stage from a base image.
<u>HEALTHCHECK</u>	Check a container's health on startup.
<u>LABEL</u>	Add metadata to an image.
<u>MAINTAINER</u>	Specify the author of an image.
<u>ONBUILD</u>	Specify instructions for when the image is used in a build.
<u>RUN</u>	Execute build commands.
<u>SHELL</u>	Set the default shell of an image.
<u>STOPSIGNAL</u>	Specify the system call signal for exiting a container.
<u>USER</u>	Set user and group ID.
<u>VOLUME</u>	Create volume mounts.
<u>WORKDIR</u>	Change working directory.

Visit for more options in the docker file

<https://docs.docker.com/reference/dockerfile/>

Working with docker container

List local images

```
sudo docker images
```

To create a container

```
docker run --name <container_name> <image_name>
```

To create a container by mounting a different drive or folder

```
sudo docker run -it --mount src=</path_to_workingdirectory_on_machine>,target=/home/,type=bind <image_name>
```

To list currently running containers:

```
sudo docker ps
```

List all docker containers (running and stopped)

```
sudo docker ps -a
```

Exiting the container

```
exit
```

Entering the container in terminal

```
sudo docker exec -it dockername /bin/bash
```

Start the container

```
docker start <CONTAINER_ID>
```

Stop the container

```
docker stop <CONTAINER_ID>
```

Removing a stopped container

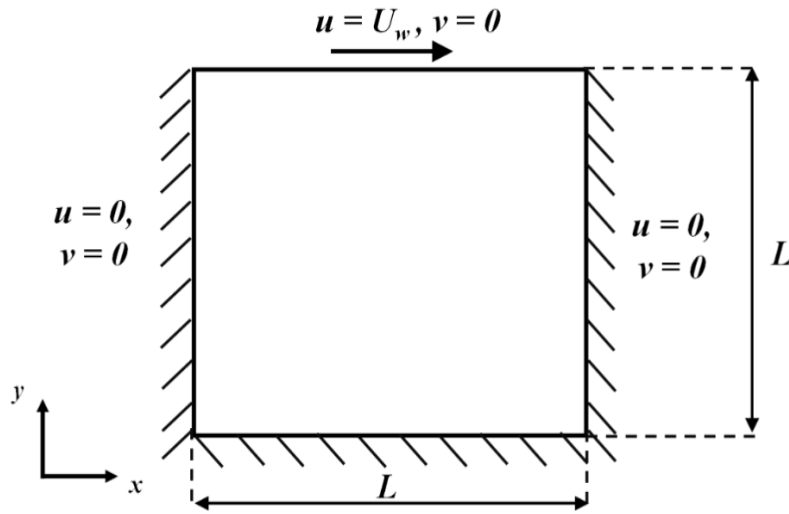
```
sudo docker rm <CONTAINER_ID>
```

For more commands

- https://docs.docker.com/get-started/docker_cheatsheet.pdf
- <https://www.geeksforgeeks.org/devops/docker-instruction-commands/>
- <https://docs.docker.com/reference/cli/docker/>

```
uqjabra1@aqua:~$ sudo docker images
REPOSITORY          TAG          IMAGE ID      CREATED      SIZE
ghcr.io/fenics/dolfinx/dolfinx  nightly      d64be49a025f  13 months ago  3.23GB
ghcr.io/marchirschvogel/ambit    devenv       570744f6d8dd  14 months ago  3.36GB
ghcr.io/marchirschvogel/ambit    latest       f85217e0aa77  14 months ago  3.4GB
hello-world             latest       74cc54e27dc4  16 months ago  10.1kB
ghcr.io/fenics/dolfinx/dolfinx  v0.9.0      cbc6e6edf697  19 months ago  2.79GB
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ ^C
uqjabra1@aqua:~$ sudo docker ps -a
CONTAINER ID   IMAGE                                COMMAND      CREATED      STATUS      PORTS      NAMES
8dae263f1019   ghcr.io/marchirschvogel/ambit       "/bin/bash"  13 months ago  Exited (137) 4 weeks ago          magical_mendeleev
5b448625718b   ghcr.io/marchirschvogel/ambit       "/bin/bash"  13 months ago  Exited (0) 13 months ago          nifty_herschel
0206f60ed51e   ghcr.io/fenics/dolfinx/dolfinx:v0.9.0 "/bin/bash"  13 months ago  Created                                pedantic_greider
6aff88a620af   ghcr.io/fenics/dolfinx/dolfinx:v0.9.0 "/bin/bash"  13 months ago  Up 13 months                        eloquent_jepsen
0c0231453a5b   ghcr.io/fenics/dolfinx/dolfinx:nightly "/bin/bash"  13 months ago  Exited (1) 13 months ago          happy_ganguly
13ed96208c6a   ghcr.io/fenics/dolfinx/dolfinx:nightly "/bin/bash"  13 months ago  Up 13 months                        busy_clarke
8d53e73915a7   ghcr.io/fenics/dolfinx/dolfinx:nightly "/bin/bash"  13 months ago  Up 13 months                        nostalgic_nightingale
9ce83e3f420d   hello-world                          "/hello"     13 months ago  Exited (0) 13 months ago          sweet_mendel
uqjabra1@aqua:~$
```

Example case: Lid driven cavity problem



$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \nu \nabla^2 \mathbf{u}$$
$$\nabla \cdot \mathbf{u} = 0$$

Apply the time discretisation: $\frac{\partial \mathbf{u}}{\partial t} \approx \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t}$

v = velocity test function
 q = pressure test function

Weak form: by multiplying with the test function, and integration by parts.

$$\int_{\Omega} \rho \left(\frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} \right) \cdot \mathbf{v} \, d\Omega + \int_{\Omega} \rho \left((\mathbf{u}^n \cdot \nabla) \mathbf{u}^{n+1} \right) \cdot \mathbf{v} \, d\Omega + \int_{\Omega} \nu \nabla \mathbf{u}^{n+1} : \nabla \mathbf{v} \, d\Omega - \int_{\Omega} p^{n+1} (\nabla \cdot \mathbf{v}) \, d\Omega + \int_{\Omega} (\nabla \cdot \mathbf{u}^{n+1}) q \, d\Omega = 0.$$

Use Taylor Hood Elements to satisfy Ladyzhenskaya–Babuška–Brezzi (LBB) stability condition for incompressible fluids.

Velocity: quadratic (P2)

Pressure: linear (P1)

Example case using FEniCS 2017-19 version

```
1 from dolfin import *
2 import matplotlib.pyplot as plt
3
4 # Define mesh and function space
5 mesh = UnitSquareMesh(50, 50)
6 P2 = VectorElement("P", mesh.ufl_cell(), 2) # Velocity element
7 P1 = FiniteElement("P", mesh.ufl_cell(), 1) # Pressure element
8 TH = MixedElement([P2, P1])
9 W = FunctionSpace(mesh, TH) #Mixed function space
10
11 # Define boundary conditions
12 inflow = Expression(("1.0", "0.0"), degree=2) # Inflow velocity
13
14 def lid(x, on_boundary):
15     return on_boundary and near(x[1], 1.0)
16
17 def walls(x, on_boundary):
18     return on_boundary and not near(x[1], 1.0)
19
20 bcu = [DirichletBC(W.sub(0), inflow, lid),
21        DirichletBC(W.sub(0), Constant((0, 0)), walls)]
```

```
23 # Time parameters
24 dt = 0.1
25 T = 2.0
26 t = 0.0 # Time step
27
28 # Functions
29 w = Function(W)
30 w_prev = Function(W)
31 u, p = split(w)
32 u_prev, p_prev = split(w_prev)
33 v, q = TestFunctions(W)
34
35 # Physical parameters
36 nu = 0.01
37 rho = 1.0
38
```

```
39 # Variational formulation (weak form of Navier Stokes equation)
40 F = (rho * inner((u - u_prev)/dt, v) * dx
41      + rho * inner(dot(u_prev, nabla_grad(u)), v) * dx
42      + nu * inner(grad(u), grad(v)) * dx
43      - p * div(v) * dx
44      + div(u) * q * dx)
45
46 # Critical fix: Compute Jacobian and don't redefine problem
47 J = derivative(F, w, TrialFunction(W))
48 problem = NonlinearVariationalProblem(F, w, bcu, J=J)
49 solver = NonlinearVariationalSolver(problem)
50 print(solver.parameters)
51 # Add solver parameters
52 prm = solver.parameters
53 prm['newton_solver']['absolute_tolerance'] = 1e-12
54 prm['newton_solver']['relative_tolerance'] = 1e-14
55 prm['newton_solver']['maximum_iterations'] = 5
56 prm['newton_solver']['linear_solver'] = 'mumps'
```

```
62 # Time-stepping
63 while t < T:
64     w_prev.assign(w)
65     solver.solve()
66     t += dt
67     #print(f"Time step t = {t:.3f}")
68
69     # Save velocity and pressure to files for visualization in ParaView
70     u, p = w.split()
71     u_file << (u, t)
72     p_file << (p, t)
73
```

Source: https://github.com/Adari-Harsha/Lid_Driven_Cavity_Flow-

Example case using dolfinx version

FEniCS (dolfin) 2019 version

```
1 from dolfin import *
2 import matplotlib.pyplot as plt
3
4 # Define mesh and function space
5 mesh = UnitSquareMesh(50, 50)
6 P2 = VectorElement("P", mesh.ufl_cell(), 2) # Velocity element
7 P1 = FiniteElement("P", mesh.ufl_cell(), 1) # Pressure element
8 TH = MixedElement([P2, P1])
9 W = FunctionSpace(mesh, TH) #Mixed function space
10
11 # Define boundary conditions
12 inflow = Expression(("1.0", "0.0"), degree=2) # Inflow velocity
13
14 def lid(x, on_boundary):
15     return on_boundary and near(x[1], 1.0)
16
17 def walls(x, on_boundary):
18     return on_boundary and not near(x[1], 1.0)
19
20 bcu = [DirichletBC(W.sub(0), inflow, lid),
21        DirichletBC(W.sub(0), Constant((0, 0)), walls)]
22
23 # Time parameters
```

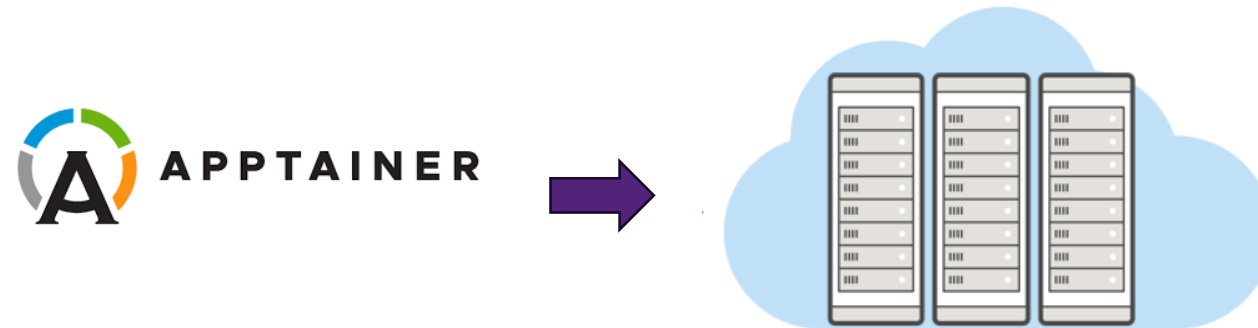
FEniCSx(dolfinx) version

```
10 from mpi4py import MPI
11 import numpy as np
12 import matplotlib.pyplot as plt
13
14 from dolfinx import mesh, fem, io
15 from dolfinx.fem import functionspace, Function, Constant, dirichletbc, locate_dofs_topological
16 from dolfinx.fem.petsc import NonlinearProblem
17 from dolfinx.io import XDMFFile
18 from basix.ufl import element, mixed_element
19 from ufl import (
20     TestFunctions,
21     split, inner, dot, grad, nabla_grad, div, dx,
22 )
23 from petsc4py import PETSc
24
25 # -----
26 # Mesh
27 # -----
28 msh = mesh.create_unit_square(MPI.COMM_WORLD, 50, 50)
29
30 # -----
31 # Mixed function space: Taylor-Hood P2/P1
32 # -----
33 P2 = element("Lagrange", msh.basix_cell(), 2, shape=(msh.geometry.dim,))
34 P1 = element("Lagrange", msh.basix_cell(), 1)
35 TH = mixed_element([P2, P1])
36 W = functionspace(msh, TH)
37
38 # -----
39 # Boundary conditions
40 # -----
41 # Locate facets on each boundary
42 tdim = msh.topology.dim
43 fdim = tdim - 1
44 msh.topology.create_connectivity(fdim, tdim)
45
46 # Lid: y == 1 (moving wall)
47 lid_facets = mesh.locate_entities_boundary(msh, fdim, lambda x: np.isclose(x[1], 1.0))
48 # Walls: all other boundaries
49 wall_facets = mesh.locate_entities_boundary(
50     msh, fdim,
51     lambda x: np.isclose(x[0], 0.0) | np.isclose(x[0], 1.0) | np.isclose(x[1], 0.0)
52 )
```

Demonstration

Part -B

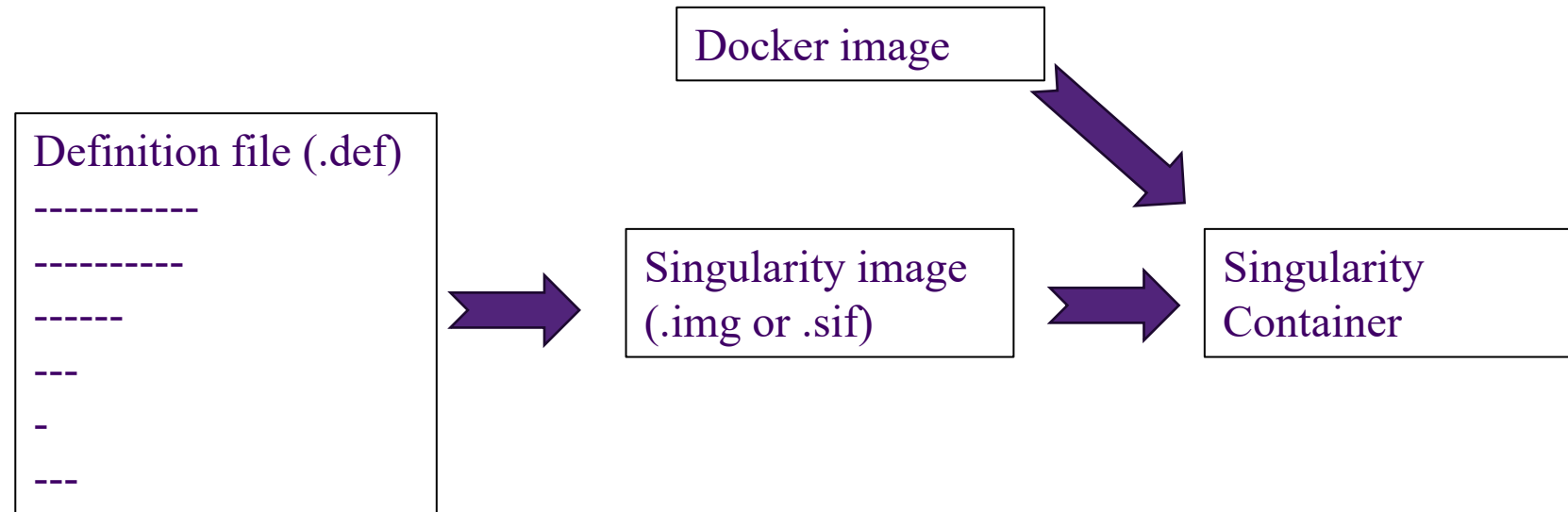
Singularity workflow



Installing singularity

To install the singularity (https://docs.sylabs.io/guides/3.5/user-guide/quick_start.html)

- Install dependencies
- Install Go
- Setup the environment
- Download singularity
- Compile the singularity using cmake



Singularity images are easily portable

Structure of singularity definition file

Header

```
Bootstrap: library
From: ubuntu:22.04
Stage: build

%setup
touch /file1
touch ${SINGULARITY_ROOTFS}/file2

%files
/file1
/file1 /opt

%environment
export LISTEN_PORT=54321
export LC_ALL=C

%post
apt-get update && apt-get install -y netcat
NOW=`date`
echo "export NOW=\"${NOW}\"" >> $SINGULARITY_ENVIRONMENT
```

Sections

```
%runscript
echo "Container was created $NOW"
echo "Arguments received: $"
exec echo "$@"

%startscript
nc -lp $LISTEN_PORT

%test
grep -q NAME="Ubuntu" /etc/os-release
if [ $? -eq 0 ]; then
    echo "Container base is Ubuntu as expected."
else
    echo "Container base is not Ubuntu."
    exit 1
fi

%labels
Author myuser@example.com
Version v0.0.1

%help
This is a demo container used to illustrate a def file that uses all
supported sections.
```

Two main parts

- Header
 - First specify the bootstrap agent to build the
 - Bootstrap options: library, docker, shub, oras, scratch....etc
 - Further entries in header are bootstrap specific
- Sections: All are optional
 - Setup
 - To be executed on host before making the OS.
 - Files
 - To copy files to the container

https://docs.sylabs.io/guides/latest/user-guide/definition_files.html

More examples: <https://github.com/sylabs/examples>

Comparison of Dockerfile and Singularity definition file

Function	Dockerfile Command	Singularity Definition File Command	Description
Base Image	FROM	BootStrap and From	Specifies the base image to use.
Labels/Metadata	LABEL	%labels	Adds metadata, such as author, version, etc.
Run Commands	RUN	%post	Executes commands during the build (e.g., installing software).
Environment Variables	ENV	%environment	Defines environment variables to be set in the container.
Default Command	CMD	%runscript	Sets the default command to run when the container is executed.
Expose Ports	EXPOSE	N/A	Docker-specific; Singularity containers don't expose ports.
Copy Files into Container	COPY or ADD	%files or cp inside %post	Copies files from the host into the container.
Working Directory	WORKDIR	cd command in %post or %runscript	Sets the working directory in the container.
Execute Command on Run	ENTRYPOINT	%runscript or exec command	Defines the main process that runs when the container starts.
Command for Tests	N/A	%test	Runs test commands to verify the build after it completes.
Custom Help Message	N/A	%help	Provides help documentation within the container.
Clean Up	RUN (e.g., apt-get clean)	Included within %post	Both involve cleaning up after installation to minimize size.
Mount Volumes	VOLUME	Bind mounts via -B flag or set in %post	Docker's VOLUME has no direct equivalent, but bind mounts are possible.

<https://mivegec.pages.ird.fr/dainat/malbec-containers/pages/containers/containers-7-singularity/#from-a-singularity-definition-file>

Examples of singularity definition file

Dockerfile

Base
image

RUN

CMD

```
E: > CFDTEA_presentation > Demonstration_files > Demonstratio
1 FROM dolfinx/dolfinx:stable
2
3 RUN apt-get update && \
4     apt-get install -y nano && \
5     rm -rf /var/lib/apt/lists/*
6
7 RUN pip install --no-cache-dir matplotlib
8
9 CMD ["/bin/bash"]
10
```

Singularity definition file

Header

Sections

```
brat / AppData / Local / temp / esdaadea-cc8b-49b3-b218-d
1 Bootstrap: docker
2 From: dolfinx/dolfinx:stable
3
4 %post
5     apt-get update && \
6     apt-get install -y nano && \
7     rm -rf /var/lib/apt/lists/*
8
9     pip install --no-cache-dir matplotlib
10
11 %runscript
12     exec /bin/bash "$@"
13
```

Building a singularity container

Building an image from the file

```
sudo singularity build <container_name>.img buildfile
```

Running an image

```
Singularity run imagename
```

Runing, it loading a directory

```
Singularity shell --bind <path to load> imagename
```

Singularity container from docker image

```
docker save -o filename.tar imagename
```

Singularity build singularityimage name.img docker-archive://imagename

```
For eg: Singularity build fenics_stable.img docker-archive://fenics_stable:py3
```

How to use singularity in Rangpur or Bunya

Keep the image in your disk space.

Point to it in your job script.

```
#!/bin/bash

#SBATCH --nodes=1
#SBATCH --ntasks=12
#SBATCH --cpus-per-task=1
#SBATCH --ntasks-per-node=6
.....

module load mpi/mpich-x86_64

mpirun singularity exec <path to the image> python filename.py
```

Demonstration

Drawbacks of container

	General drawbacks	Current options/solutions
1	Security Containers share the same host kernel, which can pose a security risk. If a container is compromised, it can potentially affect other containers on the same host.	container-specific security measures, such as container isolation and network segmentation, can mitigate this risk.
2	Complexity Containers can be complex to set up and manage, especially in large-scale environments.	Container orchestration tools, such as Kubernetes , can help to simplify the process, but they can also add complexity.
3	Storage Containers are designed to be stateless, meaning they don't store data or state. This can make managing persistent data, such as databases, difficult within a container environment.	Options such as persistent volumes, that can be used to manage data stored in a docker container environment.
4	Compatibility Containers are designed to run on a specific container runtime, such as Docker or Kubernetes. This means that applications packaged in one container format may not be compatible with another container runtime.	There are tools available, such as container conversion tools, that can be used to convert containers between different formats.

Source: https://medium.com/@xcube_LABS/the-advantages-and-disadvantages-of-containers-57458db49aa2

Thank you.....

Questions & discussion